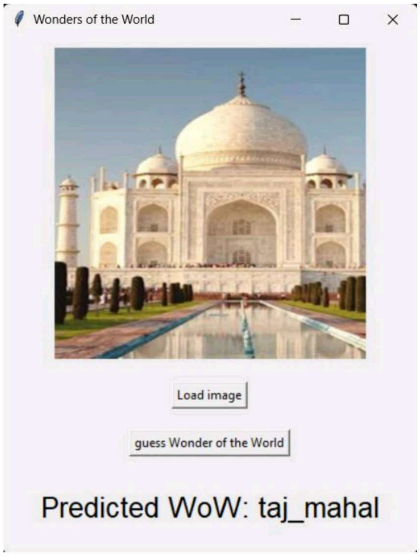
```
# train.py
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
import json

IMAGE_SIZE = (224, 224)
batch_size = 32

train_path = "C:\\Users\\HP\\Downloads\\Wonders of World\\Wonders of World\\train"
test_path = "C:\\Users\\HP\\Downloads\\Wonders of World\\Wonders of World\\test"

datagen = ImageDataGenerator(rescale=1./255)

train_ds = datagen.flow_from_directory(
    train_path,
    target_size=IMAGE_SIZE,
    batch_size=batch_size,
    class_mode='categorical'
)

val_ds = datagen.flow_from_directory(
    test_path,
    target_size=IMAGE_SIZE,
    batch_size=batch_size,
    class_mode='categorical'
)

model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(),
```

```
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(train_ds.num_classes, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(train_ds, epochs=20, validation_data=val_ds)

model.save("wow.h5")
print("Model saved successfully")

with open("labels.json", "w") as f:
    json.dump(train_ds.class_indices, f)

print("Labels saved successfully")

# test.py
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
import numpy as np
import tensorflow as tf
import json
from tensorflow.keras.preprocessing import image

model = tf.keras.models.load_model("wow.h5")

with open("labels.json", "r") as f:
    class_indices = json.load(f)
```

```
labels = list(class_indices.keys())

def predict_image(img_path):
    img = image.load_img(img_path, target_size=(224, 224))
    img = image.img_to_array(img)
    img = img / 255.0
    img = np.expand_dims(img, axis=0)

    result = model.predict(img)
    idx = np.argmax(result)

    return labels[idx]

def open_image():
    global img_path, panel

    img_path = filedialog.askopenfilename(
        filetypes=[("Image files", "*.jpg *.jpeg *.png")]
    )

    img = Image.open(img_path)
    img = img.resize((300, 300))
    img_tk = ImageTk.PhotoImage(img)

    panel.config(image=img_tk)
    panel.image = img_tk
    result.config(text="")

def classify():
    if img_path:
        label = predict_image(img_path)
        result.config(text=f"Predicted WoW: {label}")

root = tk.Tk()
root.title("Wonders of the World")
```

```
root.geometry("400x500")

panel = tk.Label(root)
panel.pack(pady=10)

btn_load = tk.Button(root, text="Load image", command=open_image)
btn_load.pack(pady=10)

btn_predict = tk.Button(root, text="guess Wonder of the World", command=classify)
btn_predict.pack(pady=10)

result = tk.Label(root, text="", font=("Arial", 20))
result.pack(pady=20)

img_path = ""

root.mainloop()
```

Outputs:

```
Epoch 1/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 188s 2s/step - accuracy: 0.3466 - loss: 1.9737 - val_accuracy: 0.5391 - val_loss: 1.4199
Epoch 2/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 121s 997ms/step - accuracy: 0.6102 - loss: 1.2162 - val_accuracy: 0.7826 - val_loss: 0.7107
Epoch 3/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 119s 984ms/step - accuracy: 0.7639 - loss: 0.7333 - val_accuracy: 0.9051 - val_loss: 0.3206
Epoch 4/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 115s 950ms/step - accuracy: 0.8887 - loss: 0.3622 - val_accuracy: 0.9587 - val_loss: 0.1635
Epoch 5/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 216s 2s/step - accuracy: 0.9649 - loss: 0.1296 - val_accuracy: 0.9855 - val_loss: 0.0569
Epoch 6/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 193s 2s/step - accuracy: 0.9878 - loss: 0.0506 - val_accuracy: 0.9986 - val_loss: 0.0144
Epoch 7/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 184s 2s/step - accuracy: 0.9969 - loss: 0.0139 - val_accuracy: 0.9971 - val_loss: 0.0194
Epoch 8/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 183s 2s/step - accuracy: 0.9875 - loss: 0.0404 - val_accuracy: 0.9949 - val_loss: 0.0214
Epoch 9/20
121/121 ━━━━━━━━━━━━━━━━━━━━ 188s 2s/step - accuracy: 0.9878 - loss: 0.0389 - val_accuracy: 0.9957 - val_loss: 0.0226
```